

LEGGETE ATTENTAMENTE QUESTE ISTRUZIONI

1. Caricate le soluzioni su <https://upload.di.unimi.it>.
2. Caricate un file per ogni esercizio, con lo stesso nome che trovate nella cartella d'esame (`es_filtro.go`, `es_1.go`, ...). File con altri nomi o formati (`txt`, `zip`, ...) saranno ignorati.
3. Allo scadere del tempo il server chiude e l'esame finisce.
4. Caricate spesso per proteggervi in caso di crash della macchina.
5. Per ritirarvi caricate un file `ritirato.go` / `ritirata.go`.

Esercizio Filtro

Scrivere un programma che legge *da riga di comando* una stringa Unicode s e un intero k , e stampi la stringa ottenuta da s sostituendo ogni simbolo con quello ottenuto spostandosi di k posizioni nella tabella Unicode. Notate che k può essere positivo, negativo o nullo. As esempio, se $k = 3$ allora ‘d’ va sostituito con ‘g’, mentre se $k = -2$ allora ‘d’ va sostituito con ‘b’.

Esempi

```
$ go run es_filtro.go abcde 1
bcdef
```

```
$ go run es_filtro.go bcdef -1
abcde
```

```
$ go run es_filtro.go èèè 10
òòò
```

```
$ go run es_filtro.go òòò -10
èèè
```

Potete verificare la correttezza della vostra soluzione così:

```
go test es_filtro.go es_filtro_test.go
```

Se l’output è di questo tipo allora la soluzione è corretta:

```
ok    command-line-arguments 0.361s
```

Se invece è di questo tipo allora è errata:

```
--- FAIL: es_filtro.go (0.00s)
    es_filtro_test.go:37:
        ...
```

Esercizio 1

Scrivete un programma che legga da riga di comando due interi non negativi n e d , e stampa il più piccolo intero ottenibile rimuovendo d cifre decimali da n . (Se n ha meno di d cifre, allora si intende che vadano rimosse tutte, pertanto il risultato è 0). A tal scopo implementate e usate una funzione

```
Rimuovi(n int) []int
```

che restituisce la lista degli interi ottenibili rimuovendo *una* cifra da n .

BONUS. Modificate la funzione in modo che abbia la seguente firma:

```
Rimuovi(n int, d int) []int
```

La funzione deve ritornare la lista di tutti gli interi ottenibili rimuovendo d cifre da n , e deve essere implementata *ricorsivamente*.

Esempio d'esecuzione:

```
$ go run sol.go 9452 2
42
```

```
$ go run sol.go 1324 3
1
```

```
$ go run sol.go 4612 1
412
```

```
$ go run sol.go 4213 3
1
```

Esercizio 2

Scrivere un programma che legge da riga di comando un intero k e, da standard input, un testo arbitrario, su una o più righe, terminato da EOF. Il programma deve poi stampare ogni parola che appare in almeno k righe, seguita da tale numero di righe. (Qui “parola” è intesa nel solito modo, cioè una stringa separata dal resto da caratteri di spaziatura). Inoltre le parole vanno stampate in ordine lessicografico. A tale scopo definite e usate una funzione

`Leggi() (righe [][]string)`

che restituisce la lista di righe lette da standard input, ognuna come lista di parole, e una funzione

`Conta(righe [][]string) map[string]int`

che, per ogni parola, calcola in quante righe appare.

Esempio d’esecuzione

```
$ go run es.go 2
ciao ciao raga ciao
ancora ciao
oggi è una bella giornata
ancora non piove
ciao !

ancora 2
ciao 3
```

Esercizio 3

Parte 1

Scrivere un programma che legga da standard input una sequenza di *punti* nel piano cartesiano, uno per riga. Ogni punto è descritto da una etichetta (una stringa) e due coordinate (numeri in virgola mobile), così:

```
etichetta;x;y
```

Definite un tipo dati `Punto` adatto a memorizzare un punto. Implementate poi una funzione `LeggiPercorso()` (`[]Punto`) che restituisce la lista di punti letta da standard input. Si assuma che l'input sia nel formato corretto e contenga almeno due punti.

Parte 2

Ogni coppia di punti consecutivi è una *tratta* del percorso. La *lunghezza* di una tratta è la distanza euclidea tra i suoi punti p e q . Ovvero, se p ha coordinate x_p, y_p e q ha coordinate x_q, y_q , allora la distanza tra p e q è:

$$d(p, q) := \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}$$

La lunghezza del percorso è la somma delle lunghezze delle sue tratte. Dopo aver letto il percorso, il programma deve stampare, arrotondando a *tre* cifre decimali:

- la lunghezza totale del percorso.
- il primo punto del percorso che si incontra dopo *almeno* metà della lunghezza totale.

A tal scopo implementate le seguenti funzioni:

- `Distanza(p, q Punto) float64` che calcola la distanza tra p e q
- `Stringa(p Punto) string` che restituisce la rappresentazione di p nel seguente formato:
`|etichetta = (x, y)|`
- `Lunghezza(P []Punto) float64` che calcola la lunghezza del percorso P

Esempio d'esecuzione:

```
$ cat P1.txt
```

```
A;10.0;2.0
```

```
B;11.5;3.0
```

```
C;8.0;1.0
```

```
D;3;4
```

```
E;1;0
```

```
F;-1;5
```

```
$ go run esercizio_3.go < P1.txt
```

```
21.522
```

```
D = (3.0, 4.0)
```

Parte 3

Modificate il programma in modo che accetti da *riga di comando* la descrizione di un ulteriore punto, sempre nel formato di input. Il programma deve aggiungere il punto al percorso letto da standard input, nella posizione che *causa il minimo incremento della lunghezza totale*. A tal scopo implementate una funzione:

```
NuovaDistanza(P []Punto, nuovoPunto Punto) []float64
```

che ritorna una slice che, per ogni $i = 0, 1, \dots$, riporta la distanza del percorso ottenuto inserendo `nuovoPunto` in posizione i nel percorso `P`. Il programma deve infine stampare, sulla stessa riga: l'etichetta del nuovo punto, la sua posizione nel nuovo percorso, e la lunghezza totale del nuovo percorso.

Esempio d'esecuzione:

```
$ cat P1.txt
A;10.0;2.0
B;11.5;3.0
C;8.0;1.0
D;3;4
E;1;0
F;-1;5

$ go run esercizio_3.go "Z;2.0;2.0" < P1.txt
21.522
D = (3.0, 4.0)
Z 4 21.751
```